

[Home](#)[GitHub](#)

## CLASSES

[ErrorBoundary](#)

## EVENTS

[SIGINT](#)[unhandledRejection](#)

## NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

## GLOBAL

[404\\_Error\\_Handler](#)[ACTIVIDADES](#)[API\\_URL](#)[ASPECTOS\\_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS\\_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)[Footer](#)[GET /](#)

# frontend/src/store/authStore.js

```
1.  /**
2.   * @file Store de autenticación con Zustand.
3.   * @description Gestiona el estado de autenticación del usuario (usu
4.   * @requires zustand
5.   * @requires zustand/middleware
6.   */
7.  import { create } from 'zustand';
8.  import { persist } from 'zustand/middleware';
9.
10. /**
11.  * @function useAuthStore
12.  * @description Hook de Zustand para acceder y manipular el estado d
13.  * @returns {object} El store de autenticación con el estado y las a
14.  * @property {object|null} user - El objeto de usuario si está auten
15.  * @property {string} token - El token JWT de autenticación.
16.  * @property {function} isAuthenticated - Una función computada que
17.  * @property {function} setAuth - Acción para guardar los datos del
18.  * @property {function} logout - Acción para limpiar los datos de au
19.  */
20. export const useAuthStore = create(
21.   persist(
22.     (set, get) => ({
23.       user: null,
24.       token: '', // Cambiado a cadena vacía para evitar 'null'
25.
26.       // Computed property para verificar autenticación
27.       isAuthenticated: () => {
28.         const state = get();
29.         return !!state.user && state.token && state.token.t
30.       },
31.
32.       // Guardar usuario y token tras login/registro
33.       setAuth: (userData, token) => {
34.         console.log('setAuth called with:', {
35.           userData: userData?.email || 'no user data',
36.           tokenExists: !!token,
37.           tokenLength: token ? token.length : 0
38.         }); // DEBUG
39.         const finalToken = token && typeof token === 'string'
40.         console.log('Saving token to store. Token length:',
41.           set(() => ({
42.             user: userData,
43.             token: finalToken,
44.           }));
45.       },
46.
47.       // Limpiar datos al hacer logout
48.       logout: () =>
49.         set(() => ({
50.           user: null,
51.           token: '', // Cambiado a cadena vacía
52.         })),
```

```
53.         }},  
54.         {  
55.             name: 'auth-storage',  
56.         }  
57.     )  
58. );
```

Documentation generated by [JSDoc 4.0.5](#) on Thu Dec 11 2025 11:56:27 GMT+0000  
(Coordinated Universal Time) using the [docdash](#) theme.